

一、程式檔案說明

knapsack_all.cpp

- 整合三種演算法，並提供輸入參數、自動生成測試數據與計時功能。
- generate_items(n, weight_min, weight_max, profit_min, profit_max)：隨機生成 n 個物品。
- knapsack_top_down(items, capacity)：Top-Down 動態規劃
- knapsack_bottom_up(items, capacity)：Bottom-Up 動態規劃
- knapsack_greedy(items, capacity)：Greedy 演算法 (依價值/重量排序後線性選擇)。
- main()：輸入測試參數 → 生成物品 → 執行三種算法 → 輸出最大價值與執行時間。

物品數量 N

背包容量 capacity

重量範圍 weight_min weight_max

價值範圍 profit_min profit_max

然後自動生成測試物品，依序執行三種算法輸出「最大價值」與「執行時間」。

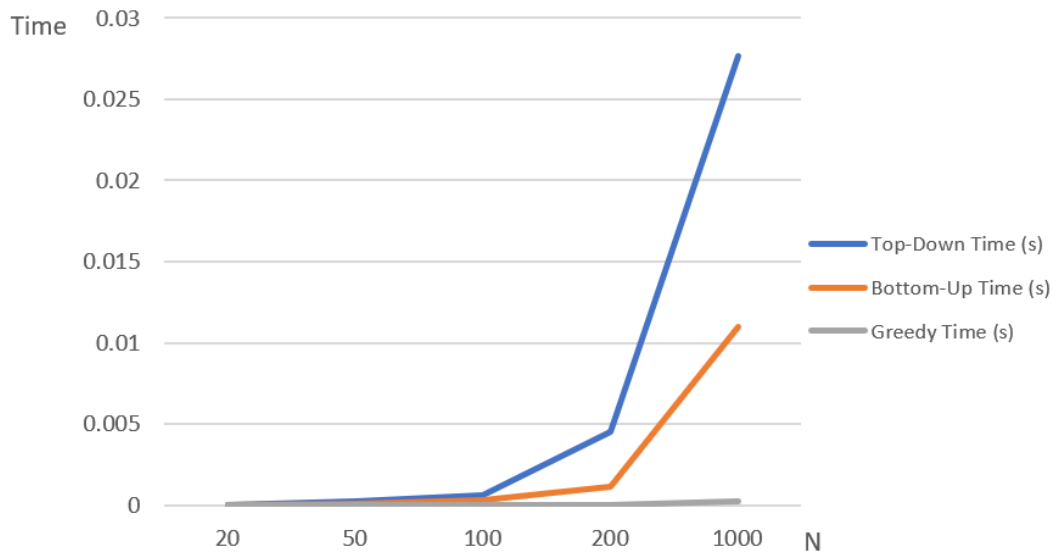
二、測試輸入設定

依據作業要求，設計五組參數來比較：

測試編號	n	Weight 範圍	Profit 範圍	Capacity
T1	20	[1,10]	[1,20]	50
T2	50	[1,20]	[10,50]	100
T3	100	[5,30]	[10,60]	200
T4	200	[10,50]	[20,100]	500
T5	1000	[1,100]	[10,200]	1000

三、實驗結果

測試編號	Top-Down Value	Top-Down Time (s)	Bottom-Up Value	Bottom-Up Time (s)	Greedy Value	Greedy Time (s)
T1	177	0.0000406	177	0.0000332	172	0.0000058
T2	693	0.0002070	693	0.0001134	664	0.0000140
T3	844	0.0005961	844	0.0003009	824	0.0000119
T4	2526	0.0045354	2526	0.0011649	2434	0.0000298
T5	16009	0.0276961	16009	0.0109540	15879	0.0002520



四、結果分析

驗證：

Top-Down 與 Bottom-Up 在所有測試中均得出相同的最大價值，證明兩種 DP 實現邏輯正確且一致。

時間效率比較：

Greedy：僅需一次排序和一次線性遍歷，其執行時間遠低於 DP 方法；即使在 $n=1000$ 、 $capacity=1000$ 時，仍能在 0.0003 秒內完成。

Bottom-Up：雙重迴圈填表，隨 n 與 $capacity$ 增大，時間呈現多項式級成長；在 T4、T5 規模下耗時約 0.001–0.01 秒。

Top-Down：雖同樣是 DP，但因遞迴呼叫與備忘錄存取開銷較大，使其在大規模測試（如 T4–T5）中比 Bottom-Up 慢約 2–3 倍。

解的誤差：

Greedy 所得解雖在所有測試中都非常接近最優值，誤差率僅在 0.7%–4.2% 之間，但仍無法保證全局最優。

當誤差可接受且時間敏感時，可優先考慮 Greedy；反之，需追求最優解時，應使用 DP 方法。

規模擴展：

在小型 ($n \leq 100$) 問題上，三者時間差距不大；但當 n 或 capacity 超過數百以上，Greedy 明顯最穩定，Bottom-Up 次之，Top-Down 則增長最為迅速。

五、結論與建議

若需保證最優解，應使用 Top-Down 或 Bottom-Up 動態規劃；若問題規模大、速度要求高，可考慮 Bottom-Up 並做空間優化。

若僅需近似解且時間敏感，Greedy 演算法因其簡單快速，可作為初步方案或結合其他啟發式方法。